

CAHIER DES CHARGES

Application SAECine

Amélioration – Sécurisation – Conteneurisation

Projet	SAECine – SAE4 2026
Groupe	Damien Rinaldis Ethan Stasse
Application	SAECine – Site web de vote pour films
Version	1.0 – Version initiale
Statut	En cours de rédaction

1. Contexte du projet

L'application SAECine est un site web développé dans le cadre du projet SAE4. Elle permet aux utilisateurs inscrits de voter pour leurs films préférés et d'interagir avec une interface d'administration dédiée.

Suite à un audit approfondi de l'existant, plusieurs déficiences ont été identifiées dans les domaines de la sécurité, des performances, de la robustesse et de la maintenabilité du code. Ce cahier des charges formalise les objectifs d'amélioration à atteindre pour livrer une application conforme aux standards professionnels.

2. Objectifs du projet

2.1 Objectifs principaux

N°	Objectif	Priorité
1	Sécuriser l'application contre les vulnérabilités connues	● Critique
2	Améliorer les performances et les temps de réponse	● Haute
3	Rendre l'application plus robuste et résistante aux erreurs	● Haute
4	Améliorer la qualité et la lisibilité du code source	● Moyenne
5	Conteneuriser l'application avec Docker	● Moyenne

2.2 Objectifs secondaires

N°	Objectif secondaire
A	Rédiger une documentation technique complète
B	Mettre en place des tests unitaires et fonctionnels
C	Améliorer l'interface d'administration
D	Optimiser la structure et les requêtes de la base de données

3. Description de l'existant

3.1 Technologies utilisées

Composant	Technologie actuelle
Langage back-end	PHP (version non maintenue)
Base de données	MySQL
Front-end	HTML / CSS / JavaScript natif
Hébergement	Serveur local ou serveur école
Versionnement	Aucun système identifié

3.2 Fonctionnalités existantes

Fonctionnalité	État	Observations
Inscription utilisateur	✓ Présente	Validation à revoir
Connexion utilisateur	✓ Présente	Pas de hashage sécurisé
Vote pour films	✓ Présente	Vote multiple possible
Interface administration	⚠ Partielle	Fonctionnalités limitées
Gestion base de données	⚠ Basique	Requêtes non sécurisées

4. Besoins fonctionnels

4.1 Gestion des utilisateurs

ID	Fonctionnalité	Description détaillée
U1	Création de compte	Formulaire d'inscription avec validation côté serveur, email unique, mot de passe fort
U2	Connexion sécurisée	Authentification avec sessions PHP sécurisées, limitation des tentatives de connexion

U 3	Modification de profil	Mise à jour des informations personnelles avec confirmation du mot de passe actuel
U 4	Réinitialisation mot de passe	Envoi d'un lien de réinitialisation par email avec token d'expiration

4.2 Gestion des votes

I D	Fonctionnalité	Description détaillée
V 1	Affichage de la liste des films	Liste paginée avec titre, affiche, synopsis et nombre de votes
V 2	Vote unique par utilisateur	Un seul vote par utilisateur par film, contrôlé côté serveur
V 3	Modification du vote	(Optionnel) Possibilité de changer son vote dans un délai défini
V 4	Affichage des résultats	Classement dynamique des films avec visualisation graphique des scores

4.3 Interface d'administration

I D	Fonctionnalité	Description détaillée
A 1	Ajout d'un film	Formulaire complet avec titre, description, affiche, année, genre
A 2	Suppression d'un film	Suppression avec confirmation et cascade sur les votes associés
A 3	Statistiques	Tableau de bord avec nombre d'utilisateurs, votes, films les plus populaires
A 4	Gestion des utilisateurs	Consultation, suspension ou suppression de comptes utilisateurs

5. Besoins non fonctionnels

5.1 Performance

Critère	Exigence	Mesure
Temps de chargement	Inférieur à 2 secondes	Lighthouse / GTmetrix
Requêtes SQL	Optimisation avec index et requêtes préparées	EXPLAIN MySQL
Mise en cache	Cache HTTP et cache applicatif PHP	Headers HTTP
Compression HTTP	Activation gzip / brotli côté serveur	DevTools navigateur

5.2 Sécurité

Mesure de sécurité	Détail de mise en œuvre
Hashage des mots de passe	Algorithme bcrypt ou argon2 via password_hash() PHP
Requêtes préparées PDO	Prévention des injections SQL sur toutes les requêtes
Protection CSRF	Tokens CSRF sur tous les formulaires POST
Protection XSS	Échappement systématique avec htmlspecialchars()
Anti brute-force	Limitation à 5 tentatives / 15 minutes par IP
HTTPS obligatoire	Redirection automatique HTTP → HTTPS, certificat SSL

5.3 Robustesse

Critère	Exigence
Gestion des erreurs	Handler global PHP avec pages d'erreur personnalisées (404, 500)
Journalisation	Logs d'erreurs structurés avec niveaux (ERROR, WARNING, INFO)
Tests unitaires	Couverture minimale de 60% des fonctions critiques avec PHPUnit
Validation serveur	Validation stricte de toutes les données reçues en entrée

5.4 Accessibilité

Critère	Exigence
Référentiel RGAA	Conformité avec le Référentiel Général d'Amélioration de l'Accessibilité
Texte alternatif	Attribut alt renseigné sur toutes les images
Contraste des couleurs	Ratio de contraste minimum 4.5:1 (WCAG AA)
Navigation clavier	Navigation complète au clavier sans souris

6. Contraintes techniques

Contrainte	Précision
Langage principal	PHP (version maintenue ≥ 8.1)
Base de données	MySQL 8.0+
Versionnement	Git obligatoire – dépôt partagé avec branches et commits réguliers
Conteneurisation	Docker + Docker Compose obligatoires
Compatibilité OS	Compatible Linux et Windows (via Docker)

7. Livrables

Phase	Livrable	Format / Support
Phase 1	Rapport d'audit de l'existant	Document PDF
Phase 1	Cahier des charges	Document Word/PDF
Phase 1	Dépôt Git initialisé et structuré	URL dépôt Git
Phase 2	Application corrigée et améliorée	Code source versionné
Phase 2	Documentation technique	Markdown / Wiki Git
Phase 2	Présentation orale du projet	Diaporama + démonstration live

8. Planning prévisionnel

Tâche	Responsable	Échéance	Statut
Audit de l'existant	Groupe complet	Semaine 1	☐ À faire
Rédaction cahier des charges	Damien Rinaldis	Semaine 1	☐ À faire
Correction des vulnérabilités	Ethan Sattse	Semaine 2	☐ À faire
Optimisation des performances	Damien Rinaldis	Semaine 2	☐ À faire
Mise en place des tests	Groupe complet	Semaine 3	☐ À faire
Conteneurisation Docker	Groupe complet	Semaine 3	☐ À faire

9. Critères de validation

Le projet sera considéré comme validé lorsque l'ensemble des critères suivants seront vérifiés et documentés :

#	Critère de validation	Méthode de vérification
C V1	Application fonctionnelle sans erreurs bloquantes	Tests manuels et automatisés
C V2	Sécurité validée (CSRF, XSS, injections SQL)	Audit sécurité / OWASP
C V3	Code documenté et maintenable	Revue de code
C V4	Tests unitaires passants (couverture $\geq 60\%$)	Rapport PHPUnit
C V5	Conteneur Docker fonctionnel (docker-compose up)	Test en environnement neutre
C V6	Rapport et livrables rendus dans les délais	Vérification par l'encadrant

10. Analyse des risques

Risque identifié	Probabilité	Impact	Mesure préventive
Manque de temps en fin de projet	● Moyenne	● Élevé	Planning clair avec jalons hebdomadaires
Apparition de bugs imprévus	● Moyenne	● Moyen	Tests réguliers, réunions d'équipe bi-hebdomadaires
Problèmes de configuration Docker	● Faible	● Moyen	Documentation officielle Docker, entraide équipe
Conflits de fusion Git	● Moyenne	● Faible	Convention de nommage des branches, commits fréquents
Perte de données / code	● Faible	● Élevé	Sauvegardes quotidiennes, dépôt Git distant

11. Conclusion

Ce cahier des charges constitue le document de référence pour le projet d'amélioration de l'application SAECine. Il formalise l'ensemble des exigences fonctionnelles, non fonctionnelles, techniques et organisationnelles que l'équipe s'engage à respecter.

Les améliorations prévues permettront de livrer une application conforme aux standards de l'industrie : sécurisée contre les principales vulnérabilités web, performante pour offrir une expérience utilisateur fluide, robuste face aux erreurs imprévues, et maintenable grâce à une base de code documentée et testée.

La mise en œuvre de Docker garantira par ailleurs la portabilité de l'application et facilitera son déploiement dans différents environnements, qu'il s'agisse des postes des développeurs ou du serveur de production de l'école.